Threads, UPS & FPS

Introduction

In this unit you are going to learn how to simulate having your program do multiple things at the same time.

You will also learn how to manage the number of frame per second and how many updates per second your program will have.

Threads

You may have noticed while running windows that several programs appear to be running simultaneously. Windows simulates running multiple programs at once by running all the programs for very short periods, one at a time.

Every running program will have one of more threads running. A **thread** is a process that manages the execution of code. A program running a single can only perform one action at a time. A program running multiple threads can perform multiple actions "simultaneously".

When you run a program it is given a thread to run your code. In this unit we are going to be building programs that have the thread they were created with, and a thread to manage how often the program paints and updates.

The first step to creating a new thread in your program is to build a file that implements Runnable. By implementing Runnable you must write a void method called run(). This method will perform the tasks you want your thread to accomplish.

Once you have a class that implements Runnable you need to create an instance of that class. Next you will create a new Thread object that takes in as a parameter your runnable instance. Finally you will call start on your Thread object; this will make the new thread that executes the run method of your Runnable class.

Example: (Assume ClassA is Runnable)

```
ClassA ca =  new ClassA();
Thread t = new Thread(ca);
t.start();
```

Managing Your Threads

Once you have created a thread it can be managed with multiple methods. Here is a list of methods you will use from the thread class:

Methods:

Return Type	Name	Description
void	start()	Starts running the thread. It will call the run method of your Runnable class.
static void	sleep(int mils)	Stops running the thread for mils milliseconds
static void	Sleep(int mils, int nanos)	Stops running the thread for mils milliseconds and nanos nanoseconds

Documentation: Resources\Thead

Pausing a Thread

When pausing a thread there is a chance it will produce an error. Because of this your compiler will not allow you to call Thread.sleep() outside of a try statement.

Here is the format for calling the sleep method:

```
try
{
        Thread.sleep(time to pause);
}
catch(Exception e)
{
        // prints the error message
        System.out.println("Error sleeping: " + e.getMessage());
}
```

Example:

Thread Examples\Thread Printing

Managing Updates per Second & Frames per Second

Updates per Second

An **update** is when a game moves items in the game world or changes values in the game.

Many old computer games did not properly manage updates per second. When the program was run on a slow computer the game would run slow. When the game was on a very good computer it would run so fast it would be unplayable.

Games are now written so that they update at the same speed even on varying computers. Unfortunately they will still run slow on really outdated computers.

Frames per Second

A **frame** is a single of image of a game video. The **frame rate** of a game is the average number of frames the game displays every second. A good frame rate is 35 frames per second or more.

A game may appear jumpy if it cannot maintain a good frame rate.

Coding Frames per Second & Updates per Second

UPS are more important than FPS. Having the correct UPS will allow the game to run at the same speed on virtually any machine. We will have a goal number for how many frames we would like to get each second, but we will **drop frames** if needed to ensure we get the correct UPS.

Example showing how to set up UPS & FPS:

Thread Examples\UPS & FPS

Terms:

Term	Definition
Dropped Frames	When frames take too long to render and some do not get painted.
FPS	The number of frames drawn per second.
Frame	A single image of a games screen.
Frame Rate	The number of frames that get drawn per second.
Render	The process of generating an image from data.
Thread	A thread is process that manages the execution of code.
Update	When the game data is updated.
UPS	The number of updates per second.